

Yann Madry

SAÉ 2.01 – Livre dont vous êtes le héros (Qt Widgets)

Suivi des tâches

Tâche	Temps	État
Analyse et conception du modèle de données (en équipe)	2 h	Terminée
Classe Page (titre, texte, image, type Normale/Victoire/Défaite, estFin)	3 h	Terminée
Classe Livre (QMap des pages, page de départ, ajout/suppression)	3 h	Terminée
versJson et depuisJson de Page et Choix	2 h	Terminée
Tests unitaires du modèle	1 h 30	Terminée
réunions d'équipe	1 h 30	Terminée
Tests et corrections	2h	Terminée
Test de sécurité : verifierCoherenceLivre()	2h	Terminée

Description d'une tâche : les classes Page, Livre et la vérification de cohérence

Fonctionnalités :

Ces deux classes constituent la structure centrale de mon code. La classe Page gère le contenu textuel, les illustrations et le type de chapitre (normal, victoire ou défaite). Elle embarque aussi la liste des choix disponibles pour le lecteur. La classe Livre, de son côté, centralise l'ensemble du projet en stockant toutes ces pages dans une QMap indexée par identifiant.

C'est cette classe Livre qui s'occupe de l'exportation finale de l'histoire. Pour cela, elle parcourt toutes les pages et génère automatiquement un fichier page_N.html pour chacune, mis en forme avec une feuille de style CSS. C'est également ici qu'on a géré (Thomas) la création de la carte interactive : l'application génère un fichier index.html qui charge la bibliothèque JavaScript Leaflet, calcule l'emplacement des marqueurs pour qu'ils soient bien espacés, et ajoute un lien vers le bon chapitre dans la bulle de chaque marqueur. Pour éviter tout problème d'affichage avec les accents, l'écriture sur le disque via QFile et QTextStream est entièrement configurée en UTF-8.

Méthode et algorithme. Pour lier les chapitres, j'ai modélisé la classe Page avec un QVector<Choix> et des accesseurs basiques, complétés par une méthode estFin() qui vérifie si le parcours s'arrête. Pour que l'utilisateur puisse sauvegarder et recharger son livre à tout moment sans rien perdre, j'ai implémenté la sérialisation JSON : chaque classe possède sa méthode versJson() (qui remplit un QJsonObject et utilise un QJsonArray pour les choix) ainsi qu'une fonction statique depuisJson() pour reconstruire fidèlement les objets.

Enfin, pour sécuriser le livre de l'auteur, j'ai développé la fonction verifierCoherenceLivre(). Elle s'appuie sur un algorithme de parcours en profondeur (DFS) codé de manière itérative (vu en graphe). J'utilise un simple QVector en guise de pile d'attente (grâce à takeLast() pour le principe LIFO), ce qui permet de simuler tous les chemins possibles à partir de la première page tout en gardant une liste des pages visitées pour bloquer les boucles infinies.

Dans cette fonction, j'ai choisi d'utiliser des opérateurs ternaires ? pour tester l'absence de titre de page. C'est une technique vue en cours de qualité de développement qui permet d'alléger le code, de gagner du temps et de le rendre plus lisible car ça évite d'écrire tout plein de (if else). Si l'algorithme trouve des pages inaccessibles ou des culs-de-sac oubliés (des pages sans choix qui ne sont ni une victoire ni une défaite), il génère une alerte claire via une QMessageBox.

Difficultés :

Le plus délicat a été de bien séparer les rôles de chaque classe pour éviter les dépendances au moment d'exporter les fichiers. La manipulation du format JSON a aussi demandé une attention particulière, principalement pour imbriquer correctement les tableaux de choix et convertir proprement le type de page (qui est une class enum) en données brutes.

Il a également été compliqué de faire le travail demandé dans l'intégralité dans les temps dédiés à cette SAE d'autant plus dans une semaine très chargée, ce qui a nécessité de trouver du temps supplémentaire de notre côté pour avancer.